

1 So Long, and Thanks for All the Seeds: Attacking 2 GGM-trees in Post-quantum signatures

3 No Author Given

4 No Institute Given

5 **Abstract. Keywords:** GGM tree · Zero-knowledge signatures · Fault
6 attacks · Post-quantum cryptography · Seed compression

7 1 Introduction

8 Digital signatures form the backbone of several aspects of modern digital life,
9 ranging from firmware updates and secure boot mechanisms to messaging applications
10 and public-key infrastructures. Their primary goal is to guarantee authenticity:
11 in principle, only the holder of the secret signing key should be able to produce
12 a valid signature for a given message. Additionally, digital signatures provide
13 integrity and non-repudiation, ensuring that a message has not been modified
14 and that the signer cannot later deny having produced the signature. Because
15 of their widespread deployment in security-critical systems, the compromise of
16 a digital signature scheme may have severe consequences, including malicious
17 software updates, device impersonation, and unauthorized access to sensitive
18 communications.

19 The security of classical signature schemes, such as RSA-based signatures and
20 elliptic-curve-based signatures relying on the Elliptic Curve Discrete Logarithm
21 Problem (ECDLP), would be compromised by a large quantum computer through
22 the use of Shor’s algorithm. This threat has motivated the development of
23 post-quantum cryptography, whose objective is to design cryptographic schemes
24 resistant to both classical and quantum adversaries. Among the most promising
25 candidates are zero-knowledge-based signature schemes obtained via the Fiat–
26 Shamir transformation [9,10] of interactive identification protocols. Several recent
27 constructions, including LESS [2], CROSS [3], and MEDS [8,7], rely on the
28 Fiat–Shamir transform to construct digital signatures. However, these schemes
29 typically suffer from large signature sizes. To reduce this issue, they employ
30 GGM-tree-like seed-compression techniques, which reduce the signature size by
31 compacting it to large collections of ephemeral randomness.

32 During signing, these schemes reveal only the seeds associated with public rounds
33 while keeping challenge-dependent seeds hidden. Although this mechanism is
34 essential for efficiency, recent works have shown that it also introduces a significant
35 fault-attack surface: faults affecting the seed-publication logic may leak hidden
36 seeds together with their corresponding zero-knowledge responses, enabling recovery
37 of secret information and, in the end, full key recovery or forgery attacks [14,13].

1.1 Related Work

Several recent works have studied fault attacks against post-quantum signature schemes relying on GGM-tree or seed-tree compression techniques, including LESS [2], CROSS [3], MEDS [8], and MPC-in-the-Head-based schemes [6,5].

In [13], the authors introduce fault attacks against the seed-tree compression mechanism used in zero-knowledge-based code-based signatures, primarily targeting LESS-v1 and extending the analysis to CROSS and MEDS. By faulting the seed-publication process, the attacks disclose hidden ephemeral seeds and enable recovery of secret information.

The work of [14] revisits these attacks for LESS-v2, whose incomplete GGM-tree invalidates the original attack strategy. The authors identify new fault-injection points in the incomplete tree-generation and seed-publication procedures, showing that recovering secret-equivalent information is sufficient for universal forgery.

More recently, [12] proposes an alternative attack on LESS-v2 that avoids targeting the tree structure itself. Instead, the attack exploits iteration-skip and loop-abort faults in the preprocessing step that transforms the digest vector before tree generation, enabling high-probability full key recovery.

Our work unifies these attacks under a single Generic ZK Seed Tree (GZKST) abstraction, formalizing the correctness and seed-hiding invariants implicitly exploited by prior works, while additionally providing practical fault attacks against MEDS.

Our Contributions. This work provides a unified analysis of fault attacks against post-quantum signature schemes relying on seed-tree compression mechanisms. Our main contributions are as follows:

Still need to check
this contributions

- **Generic Seed-Tree Abstraction.** We introduce the *Generic ZK Seed Tree* (GZKST) model, a unified abstraction capturing the seed-tree generation used in schemes such as LESS-v2, MEDS, and CROSS.
- **Unified Analysis of Existing Attacks.** We show that previous fault attacks against LESS-v1 and LESS-v2 can be interpreted as concrete violations of the same seed-hiding invariant, despite targeting different implementation layers and tree constructions.
- **Query Complexity Analysis.** We provide bounds on the number of effective faulted signatures required for complete secret recovery and show that, for all MEDS parameter sets, only a small number of successful queries is sufficient in practice.
- **Practical Fault Injection Attacks on MEDS.** We present practical clock-glitch fault injection attacks against the MEDS reference implementation on an ARM Cortex-M4 microcontroller using a ChipWhisperer-Lite platform. We identify multiple exploitable fault surfaces in the tree traversal logic, including complete and partial disclosure of the seed tree.

79 2 Background

80 2.1 GGM trees

81 *Pseudorandom functions.* A central problem in cryptography is generating randomness
 82 that appears truly random while remaining reproducible from a secret key. A
 83 **pseudorandom function** (PRF) addresses this problem: it is a keyed function
 84 that, to any efficient adversary without the key, is indistinguishable from a
 85 truly random function. Intuitively, a PRF expands a short secret key into an
 86 exponentially large table of “random-looking” outputs without storing the table
 87 explicitly.

88 PRFs are fundamental primitives in symmetric cryptography and appear in the
 89 construction of message authentication codes (MACs), stream ciphers, and key-
 90 derivation functions. A standard construction of PRFs from one-way functions is
 91 the **Goldreich–Goldwasser–Micali (GGM) tree** [11], which builds a PRF
 92 by iteratively applying a pseudorandom generator along the branches of a binary
 93 tree indexed by the input bits. GGM trees are the main PRF construction studied
 94 in this work.

95 **Definition 1 (GGM Tree).** Let $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ be a length-doubling
 96 pseudorandom generator, decomposed as $G(x) = (G_0(x), G_1(x))$, where G_0 and
 97 G_1 each output λ bits. Given a root seed $\text{seed}_r \in \{0, 1\}^\lambda$ and a depth d , the GGM
 98 tree is a complete binary tree of depth d in which:

- 99 – the root holds seed_r ;
- 100 – each internal node v storing seed s generates its left child as $G_0(s)$ and its
- 101 right child as $G_1(s)$;
- 102 – the 2^d leaves are the G outputs at depth d .

103 The crucial property exploited by ZK signature schemes is the *punctured-PRF*
 104 *property*: given all leaf seeds *except* leaf i , it is computationally infeasible to
 105 recover the seed at leaf i , provided G is a secure pseudorandom generator. This
 106 makes the GGM tree suitable for hiding one or more seeds while revealing the
 107 rest, as shown in Figure 1.

108 *Incomplete GGM trees.* When the number of required leaves t is not a power
 109 of 2, a scheme may use an *incomplete* tree: leaves appear at multiple levels,
 110 and internal nodes near the bottom may have fewer than two children or act
 111 as leaves themselves. This reduces memory and computation, but the additional
 112 index-tracking logic introduces new fault surfaces.

113 2.2 Attack targets

114 This section provides an overview of the signature schemes, to which we apply
 115 our generalized model and against which we carry out our attacks.

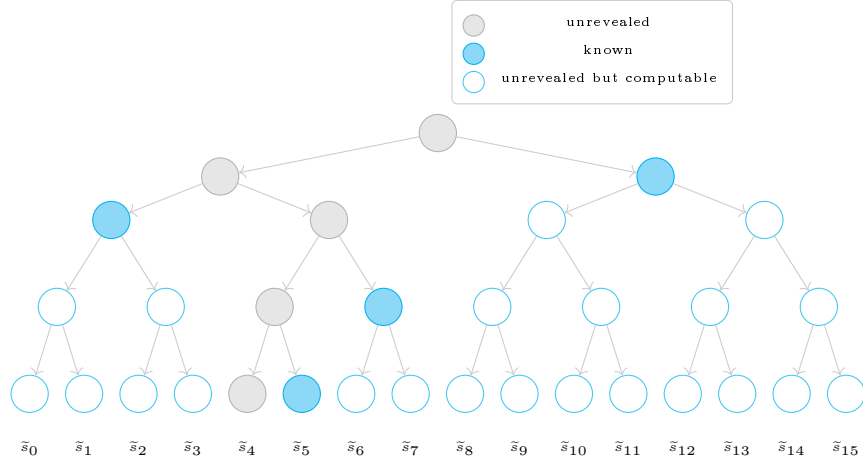


Fig. 1: Receiver’s GGM tree of depth 4. Given only the blue nodes and the PRNG function, the receiver can retrieve all leaves except x_4 .

116 **LESS-v2.** LESS (Linear Equivalence Signature Scheme) [2] uses a Fiat–Shamir
 117 transformation over a 3-pass Sigma protocol whose security rests on the Canonical
 118 Form Linear Equivalence Problem (CF-LEP). During signing, the signer must
 119 generate t independent ephemeral monomial matrices $\mathbf{Q}_{e_0}, \dots, \mathbf{Q}_{e_{t-1}} \in \text{Mon}_n(q)$.

120 *Role of the GGM tree in signing and verification.* Each leaf $\tilde{s}_i$, for $0 \leq i < t$,
 121 is an ephemeral seed of length λ bits from which the i -th monomial map $\mathbf{Q}_{e_i} \in$
 122 $\text{Mon}_n(q)$ is derived by expanding $\tilde{s}_i$ through a PRNG. The tree thus compactly
 123 encodes all t independent ephemeral matrices required by the iterated Sigma
 124 protocol from a single master seed $\text{mseed} \in \{0, 1\}^\lambda$.

125 During signing, the Fiat–Shamir challenge vector $\mathbf{b} = (b[0], \dots, b[t-1]) \in \mathcal{S}_{t,w}$
 126 partitions the rounds into two sets: the w rounds with $b[i] \neq 0$, for which the
 127 signer publishes $\text{CosetRep}(\pi_{b[i]} \circ \tilde{\pi}_i^*)$ and must *conceal* $\tilde{s}_i$, and the remaining $t-w$
 128 rounds, for which $\tilde{s}_i$ itself is disclosed so that the verifier can re-derive $\mathbf{Q}_{e_i}$ and
 129 re-compute the commitment $\text{CF}(A_i)$.

130 On the verifier side, each received node is expanded downward by the PRNG
 131 until all required leaves are reconstructed; the hidden leaves, corresponding to
 132 rounds with $b[i] \neq 0$, are never reachable from any published node, so the
 133 ephemeral seeds $\tilde{s}_i$ remain protected.

134 *Secret-extraction opportunity.* For every round i with $b[i] \neq 0$, the signer publishes
 135 $\text{CosetRep}(\pi_{b[i]} \circ \tilde{\pi}_i^*)$ while keeping $\tilde{s}_i$ hidden. If an attacker obtains both values
 136 simultaneously, Theorem 2 of [14] shows that the secret permutation $\pi_{b[i]}$ is
 137 recoverable. The flag tree is the *sole enforcement mechanism* that prevents this
 138 dual revelation. Table 1 lists the LESS-v2 parameter sets.

CROSS. CROSS (Codes and Restricted Objects Signature Scheme) [3] is based on the Restricted Syndrome Decoding Problem (R-SDP) [4] and applies the Fiat–Shamir transform to a 5-pass (q_2 -identification) protocol. Like MEDS, CROSS uses a *binary* fixed-weight challenge vector $\mathbf{c} = (c_0, \dots, c_{t-1}) \in \{0, 1\}^t$ of weight w , so the hidden and revealed index sets are $\mathcal{H} = \{i : c_i = 1\}$ and $\mathcal{R} = \{i : c_i = 0\}$, identical in structure to MEDS.

Role of the seed tree in signing and verification. Each leaf Seed_i , for $0 \leq i < t$, is a λ -bit ephemeral seed from which the pair of randomness vectors $(\mathbf{e}'_i, \mathbf{u}'_i) \in G \times \mathbb{F}_p^n$ is derived via $\text{CSPRNG}(\text{Seed}_i)$. These vectors are the sole secret material for round i : from them the signer computes the linear transitive map $\mathbf{v}_i = \mathbf{e} \star (\mathbf{e}'_i)^{-1} \in G$ and the two commitments $\text{cmt}_0[i] = \text{Hash}((\mathbf{v}_i \star \mathbf{u}'_i) \mathbf{H}^\top \mid \mathbf{v}_i)$ and $\text{cmt}_1[i] = \text{Hash}(\mathbf{u}'_i \mid \mathbf{e}'_i)$. The seed tree thus compactly encodes all t pairs of ephemeral vectors required by the iterated Sigma protocol from a single master seed.

During signing, the Fiat–Shamir second challenge vector $\text{chall}_2 \in \mathcal{B}(t, w)$ of fixed Hamming weight w partitions the rounds into two sets: the w rounds with $\text{chall}_2[i] = 1$, for which the signer publishes the explicit response $\text{resp}[i] = (\mathbf{y}_i, \mathbf{v}_i)$ and must *conceal* Seed_i , and the remaining $t - w$ rounds with $\text{chall}_2[i] = 0$, for which Seed_i itself is disclosed so that the verifier can re-expand $(\mathbf{e}'_i, \mathbf{u}'_i)$, recompute $\mathbf{y}_i = \mathbf{u}'_i + \text{chall}_1[i] \cdot \mathbf{e}'_i$, and recover $\text{cmt}_1[i]$.

Additionally, CROSS uses a parallel *Merkle tree* over the t commitments $\text{cmt}_0[i]$ to avoid transmitting all t digests explicitly: for the w rounds where $\text{chall}_2[i] = 1$, the corresponding $\text{cmt}_0[i]$ values cannot be recomputed by the verifier (since Seed_i is hidden), and are instead authenticated via a Merkle proof of size $O(w \log t)$.

On the verifier side, the received seed-path nodes are expanded downward to reconstruct all unrevealed leaf seeds; the w hidden seeds, corresponding to rounds with $\text{chall}_2[i] = 1$, are provably unreachable from any published node, ensuring that the ephemeral vectors $(\mathbf{e}'_i, \mathbf{u}'_i)$ — and hence the secret key $\mathbf{e}$ — remain protected.

Secret-extraction opportunity. For round i with $c_i = 1$, the signer reveals a ZK response encoding the relationship between $\tilde{s}_i$ and the long-term secret R-SDP solution. If an attacker obtains both the response and $\tilde{s}_i$, the secret is recoverable by the linear-algebra argument of ZKFault [13], which demonstrated full key recovery from a single fault for all CROSS parameter sets (see Table 1).

MEDS. MEDS (Matrix Equivalence Digital Signature) [8] is a code-based signature scheme whose security rests on the Matrix Code Equivalence Problem (MCEP). Like LESS, it applies the Fiat–Shamir transform to a 3-pass Sigma protocol between a prover holding a secret equivalence $(A, B) \in \text{GL}_k(q) \times \text{GL}_n(q)$ and a verifier.

Role of the seed tree in signing and verification. Each leaf σ_i , for $0 \leq i < t$, is an ephemeral seed of length $\ell_{\text{tree_seed}}$ bytes from which the i -th pair of

Table 1: Cryptographic parameters for LESS-v2 [2], MEDS [8], and CROSS [3] (*: MEDS uses $n=m=k$; CROSS uses $q=2$ implicitly via $\mathbb{F}_p$).

Scheme	Parameter set	t	w	s	Sec.	Extra
LESS-v2	LESS-252-45	45	34	8	I	$n=252, k=126, q=127$
	LESS-252-68	68	42	4	I	$n=252, k=126, q=127$
	LESS-252-192	192	36	2	I	$n=252, k=126, q=127$
	LESS-400-102	102	61	4	III	$n=400, k=200, q=127$
	LESS-400-220	220	68	2	III	$n=400, k=200, q=127$
	LESS-548-137	137	79	4	V	$n=548, k=274, q=127$
	LESS-548-345	345	75	2	V	$n=548, k=274, q=127$
MEDS*	MEDS-9923	1152	14	4	I	$n=14, q=4093$
	MEDS-13220	192	20	5	I	$n=14, q=4093$
	MEDS-41711	608	26	4	III	$n=22, q=4093$
	MEDS-69497	160	36	5	III	$n=22, q=4093$
	MEDS-134180	192	52	5	V	$n=30, q=2039$
	MEDS-167717	112	66	6	V	$n=30, q=2039$
CROSS	CROSS-R-SDP-1	163	60	2	I	$p=127, n=68, k=8$
	CROSS-R-SDP-3	200	75	2	III	$p=127, n=80, k=9$
	CROSS-R-SDP-5	250	94	2	V	$p=127, n=90, k=9$
	CROSS-R-SDPe-1	55	25	2	I	$p=509, n=60, k=6$
	CROSS-R-SDPe-3	75	34	2	III	$p=509, n=80, k=8$
	CROSS-R-SDPe-5	90	40	2	V	$p=509, n=90, k=8$

ephemeral invertible matrices $(\tilde{A}_i, \tilde{B}_i) \in \text{GL}_m(q) \times \text{GL}_n(q)$ is sampled via a PRNG. The commitment for round i is then $h_i = H(\text{SF}(\pi_{\tilde{A}_i, \tilde{B}_i}(G_0)))$, so the seed tree compactly encodes all t ephemeral matrix pairs required by the iterated Sigma protocol from a single master seed $\rho_{0,0} \in \mathcal{B}^{\ell_{\text{tree_seed}}}$.

During signing, the Fiat-Shamir challenge string $\mathbf{h} = (h_0, \dots, h_{t-1}) \in \{0, \dots, s-1\}^t$ of fixed weight w partitions the rounds into two sets: the w rounds with $h_i \neq 0$, for which the signer publishes the coset representative $(\mu_i, \nu_i) = (A\tilde{A}_i^{-1}, B^{-1}\tilde{B}_i)$ and must *conceal* σ_i , and the remaining $t-w$ rounds with $h_i = 0$, for which σ_i is disclosed so that the verifier can re-expand $(\tilde{A}_i, \tilde{B}_i)$ and recover the commitment h_i .

On the verifier side, the required leaf seeds are recovered; the w hidden seeds, corresponding to rounds with $h_i \neq 0$, are provably unreachable from any published node.

Since we are going to show in §4 an application of the attack in the reference implementation of MEDS, we are going to explain better the algorithms of key generation, signing and verification. Table 1 shows the parameters for MEDS [8].

Key Generation. Algorithm 1 is the simplification of the key generation. It begins by sampling a master secret seed δ and immediately deriving two sub-

Algorithm 1: MEDS Key Generation (simplified)

Input: —
Output: Public key pk, secret key sk

```

1   $\delta \xleftarrow{\$} \mathcal{B}^{\ell_{\text{sec\_seed}}}$ 
2   $\sigma_{G_0}, \sigma \leftarrow \text{XOF}(\delta, \ell_{\text{pub\_seed}}, \ell_{\text{sec\_seed}})$ 
3   $G_0 \leftarrow \text{ExpandSystMat}(\sigma_{G_0})$ 
4  for  $i = 1$  to  $s - 1$  do
5       $\sigma_a, \sigma_{T_i}, \sigma \leftarrow \text{XOF}(\sigma, \ell_{\text{sec\_seed}}, \ell_{\text{sec\_seed}}, \ell_{\text{sec\_seed}})$ 
6       $T_i \leftarrow \text{ExpandInvMat}(\sigma_{T_i}, k)$ 
7       $a_{m-1, m-1} \leftarrow \text{ExpandFqs}(\sigma_a, 1)$ 
8       $G'_0 \leftarrow T_i G_0$ 
9       $(A_i, B_i^{-1}) \leftarrow \text{Solve}(G'_0, a_{m-1, m-1})$  // Fail  $\Rightarrow$  retry from line 5
10      $G_i \leftarrow \text{SF}(\pi_{A_i, B_i}(G_0))$  //  $\perp \Rightarrow$  retry from line 5
11 pk  $\leftarrow (\sigma_{G_0} | \text{CompressG}(G_1) | \dots | \text{CompressG}(G_{s-1}))$ 
12 sk  $\leftarrow (\delta | \sigma_{G_0} | \text{Compress}(A_1^{-1}) | \dots | \text{Compress}(A_{s-1}^{-1}) | \text{Compress}(B_1^{-1}) | \dots | \text{Compress}(B_{s-1}^{-1}))$ 
13 return pk, sk

```

198 seeds via XOF: a *public* seed σ_{G_0} , used to expand the base systematic matrix
199 $G_0 \in \mathbb{F}_q^{k \times mn}$, and a *private* chaining seed σ that is used in the rest of the
200 process. For each of the $s - 1$ public keys, the algorithm performs a *compressed*
201 *key-generation* step: a secret invertible matrix $T_i \in \text{GL}_k(q)$ twists G_0 into $G'_0 =$
202 $T_i G_0$, and a structured linear system (the Solve step, made efficient by fixing
203 $P_0 = I_m$ and $P_1 = U_m$) recovers the equivalence pair $(A_i, B_i) \in \text{GL}_m(q) \times \text{GL}_n(q)$
204 satisfying $G_i = \text{SF}(\pi_{A_i, B_i}(G_0))$. The public key stores only the seed σ_{G_0} and the
205 compressed matrices $G_1, \dots, G_{s-1}$; the secret key retains δ together with the
206 stored inverses A_i^{-1}, B_i^{-1} —the sole secret material consumed at signing time.

207 *Signing.* Algorithm 2 is the simplification of the signing algorithm. A fresh
208 master seed δ is drawn and expanded into a tree-root seed ρ and a salt α ; the
209 seed tree $\text{SeedTree}_t(\rho, \alpha)$ then materialises all t leaf seeds $\sigma_0, \dots, \sigma_{t-1}$ at once.
210 For every round i , a pair of ephemeral matrices $(\tilde{A}_i, \tilde{B}_i) \in \text{GL}_m(q) \times \text{GL}_n(q)$ is
211 expanded from σ_i (together with α and an address for domain separation), and
212 the commitment $\tilde{G}_i = \text{SF}(\pi_{\tilde{A}_i, \tilde{B}_i}(G_0))$ is computed. Hashing all non-systematic
213 parts of the $\tilde{G}_i$ alongside the message yields the digest d , which is parsed into the
214 fixed-weight challenge vector $(h_0, \dots, h_{t-1})$. For each *non-zero* challenge h_i , the
215 signer releases the coset representative $(\mu_i, \nu_i) = (\tilde{A}_i A_{h_i}^{-1}, B_{h_i}^{-1} \tilde{B}_i)$; the remaining
216 seeds are transmitted compactly via a seed-tree path p .

217 *Verification.* Algorithm 3 is the simplification of the verification algorithm. The
218 verifier first reconstructs the public matrices $G_0, G_1, \dots, G_{s-1}$ from the public
219 key and parses the signed message to recover the path p , the digest d , the salt
220 α , and the w response pairs. The challenge vector $(h_0, \dots, h_{t-1})$ is recomputed
221 from d , and PathToSeedTree_t recovers the $t - w$ disclosed leaf seeds from p .
222 For each round i , one of two paths is taken: if $h_i > 0$, the verifier applies
223 the published coset representative (μ_i, ν_i) to the corresponding public matrix
224 G_{h_i} and checks invertibility before computing $\tilde{G}_i = \text{SF}(\pi_{\mu_i, \nu_i}(G_{h_i}))$; if $h_i = 0$,

Algorithm 2: MEDS Signing (simplified)

Input: Secret key sk , message m
Output: Signed message m_s

```

1 Parse  $sk$  to recover  $\sigma_{G_0}$ , and  $A_i^{-1}$ ,  $B_i^{-1}$  for  $i = 1, \dots, s-1$ 
2  $G_0 \leftarrow \text{ExpandSystMat}(\sigma_{G_0})$ 
3  $\delta \xleftarrow{\$} \mathcal{B}^{\ell_{\text{sec\_seed}}}$ 
4  $\rho, \alpha \leftarrow \text{XOF}(\delta, \ell_{\text{tree\_seed}}, \ell_{\text{salt}})$ 
5  $(\sigma_0, \dots, \sigma_{t-1}) \leftarrow \text{SeedTree}_t(\rho, \alpha)$ 
6 for  $i = 0$  to  $t-1$  do
7    $\sigma'_i \leftarrow (\alpha | \sigma_i | \text{ToBytes}(2^{1+\lceil \log_2 t \rceil} + i, 4))$ 
8    $\sigma_{\hat{A}_i}, \sigma_{\hat{B}_i} \leftarrow \text{XOF}(\sigma'_i, \ell_{\text{pub\_seed}}, \ell_{\text{pub\_seed}})$ 
9    $\hat{A}_i \leftarrow \text{ExpandInvMat}(\sigma_{\hat{A}_i}, m); \hat{B}_i \leftarrow \text{ExpandInvMat}(\sigma_{\hat{B}_i}, n)$ 
10   $\tilde{G}_i \leftarrow \text{SF}(\pi_{\hat{A}_i, \hat{B}_i}(G_0)) \quad // \perp \Rightarrow \text{resample } \sigma'_i$ 
11   $d \leftarrow H(\text{Compress}(\tilde{G}_0[; k, mn-1]) \parallel \dots \parallel \text{Compress}(\tilde{G}_{t-1}[; k, mn-1]) \parallel m)$ 
12   $(h_0, \dots, h_{t-1}) \leftarrow \text{ParseHash}_{s,t,w}(d)$ 
13  for  $i = 0$  to  $t-1$  with  $h_i > 0$  do
14     $\mu_i \leftarrow \hat{A}_i \cdot A_{h_i}^{-1}; \nu_i \leftarrow B_{h_i}^{-1} \cdot \hat{B}_i$ 
15    Append  $(\text{Compress}(\mu_i) \parallel \text{Compress}(\nu_i))$  to response vector  $v$ 
16   $p \leftarrow \text{SeedTreeToPath}_t(h_0, \dots, h_{t-1}, \rho, \alpha)$ 
17 return  $m_s \leftarrow (v \parallel p \parallel d \parallel \alpha \parallel m)$ 

```

225 the verifier re-expands $(\hat{A}_i, \hat{B}_i)$ from the recovered seed and computes $\hat{G}_i =$
226 $\text{SF}(\pi_{\hat{A}_i, \hat{B}_i}(G_0))$. Finally, the recomputed digest d' is compared to d ; equality
227 confirms the signature.

228 *Secret-extraction opportunity.* For round i with $h_i = 1$, the signer reveals its ZK
229 response. If an attacker obtains both the response and $\tilde{s}_i$, the secret equivalence
230 pair (A, B) is recoverable by linear algebra. The Reference Tree is the sole
231 enforcement mechanism preventing this dual revelation.

232 **3 Abstract Model**

233 We now define a unified abstraction that captures the common structure of the
234 GGM-tree mechanism across the schemes discussed in Section 2.

235 **Definition 2 (GZKST).** A Generic ZK Seed Tree (GZKST) is a tuple $\Pi =$
236 $(\text{Setup}, \text{Expand}, \text{Chal}, \text{Decide}, \text{Publish}, \text{Recon})$ parametrised by:

- 237 – λ : the security parameter;
238 – $t \in \mathbb{N}$: the number of parallel ZK repetitions;
239 – $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2^\lambda}$: a secure pseudorandom generator;
240 – $\mathcal{S}$: the secret type (e.g., permutations for LESS, matrix pairs for MEDS,
241 party shares for MPCitH).

242 The six algorithms are:

- 243 1. $\text{Setup}(1^\lambda) \rightarrow (\text{seed}_r, \text{salt})$: samples a master seed $\text{seed}_r \xleftarrow{\$} \{0, 1\}^\lambda$ and a
244 public salt.

Algorithm 3: MEDS Verification (simplified)

Input: Public key pk , signed message m_s
Output: Message m or $\perp$

```

1  $G_0 \leftarrow \text{ExpandSystMat}(\text{pk}[0, \ell_{\text{pub\_seed}} - 1])$ 
2 for  $i = 1$  to  $s - 1$  do
3    $G_i \leftarrow \text{DecompressG}(\text{pk}[\dots])$ 
4 Parse  $m_s$  to obtain: response pairs  $((\mu_i, \nu_i))_{h_i > 0}$ , path  $p$ , digest  $d$ , salt  $\alpha$ , message  $m$ 
5  $(h_0, \dots, h_{t-1}) \leftarrow \text{ParseHash}_{s,t,w}(d)$ 
6  $(\sigma_0, \dots, \sigma_{t-1}) \leftarrow \text{PathToSeedTree}_t(h_0, \dots, h_{t-1}, p, \alpha)$ 
7 for  $i = 0$  to  $t - 1$  do
8   if  $h_i > 0$ 
9      $(\mu_i, \nu_i) \leftarrow \text{next response pair from } m_s$ 
10    if  $\mu_i \notin \text{GL}_m(q)$  or  $\nu_i \notin \text{GL}_n(q)$  return  $\perp$ 
11     $\hat{G}_i \leftarrow \text{SF}(\pi_{\mu_i, \nu_i}(G_{h_i}))$ 
12    if  $\hat{G}_i = \perp$  return  $\perp$ 
13  else
14     $\sigma'_i \leftarrow (\alpha | \sigma_i | \text{ToBytes}(2^{1+\lceil \log_2 t \rceil} + i, 4))$ 
15     $\hat{A}_i \leftarrow \text{ExpandInvMat}(\text{XOF}(\sigma'_i)_1, m)$ ;  $\hat{B}_i \leftarrow \text{ExpandInvMat}(\text{XOF}(\sigma'_i)_2, n)$ 
16     $\hat{G}_i \leftarrow \text{SF}(\pi_{\hat{A}_i, \hat{B}_i}(G_0))$ 
17  $d' \leftarrow H(\text{Compress}(\hat{G}_0[k, mn-1]) \dots \text{Compress}(\hat{G}_{t-1}[k, mn-1]) | m)$ 
18 if  $d = d'$  return  $m$ 
19 return  $\perp$ 

```

- 245 2. $\text{Expand}(\text{seed}_r, \text{salt}) \rightarrow (\mathcal{T}, (\tilde{s}_0, \dots, \tilde{s}_{t-1}))$: builds a binary tree $\mathcal{T}$ by top-down
246 G-expansion from seed_r . Each leaf $i \in \{0, \dots, t-1\}$ holds a seed $\tilde{s}_i$. Tree
247 topology may be complete or incomplete (scheme-dependent).
248 3. $\text{Chal}(\text{cmt}, \text{msg}) \rightarrow \mathbf{c} = (c_0, \dots, c_{t-1})$: derives a Fiat-Shamir challenge from
249 the commitment and message. $c_i = 0$ indicates that round i requires the
250 ephemeral seed; $c_i \neq 0$ indicates a secret-dependent response.
251 4. $\text{Decide}(\mathbf{c}) \rightarrow (\mathcal{H}, \mathcal{R})$: partitions $\{0, \dots, t-1\}$ into the hidden index set $\mathcal{H} =$
252 $\{i : c_i \neq 0\}$ and the revealed index set $\mathcal{R} = \{i : c_i = 0\}$.
253 5. $\text{Publish}(\mathcal{T}, \mathcal{H}) \rightarrow \text{path}$: constructs a flag structure $\mathcal{F} : V(\mathcal{T}) \rightarrow \{0, 1\}$ and
254 returns the minimal set of nodes path from which every $\tilde{s}_i$ with $i \in \mathcal{R}$ is
255 reconstructible, while no $\tilde{s}_i$ with $i \in \mathcal{H}$ is derivable.
256 6. $\text{Recon}(\text{path}, \mathbf{c}, \text{salt}) \rightarrow \{\tilde{s}_i : i \in \mathcal{R}\}$: verifier-side reconstruction of revealed
257 leaf seeds from path .

258 **3.1 The Flag Structure**

259 The flag structure $\mathcal{F}$ produced internally by **Publish** is the key intermediate
260 object. It is a labelling $\mathcal{F} : V(\mathcal{T}) \rightarrow \{0, 1\}$ where:

- 261 – $\mathcal{F}(v) = 0$: node v is *publishable*—either directly included in **path** or derivable
262 from a published ancestor;
263 – $\mathcal{F}(v) = 1$: node v is *hidden*—it must not appear in **path** and must not be
264 derivable from published nodes.

265 The flag structure satisfies two invariants:

Invariant 1 (Correctness). For every $i \in \mathcal{R}$ there exists a node v on the path from the root to leaf $\tilde{s}_i$ such that $\mathcal{F}(v) = 0$ and $\mathcal{F}(\text{parent}(v)) = 1$.

Invariant 2 (ZK hiding). For every $i \in \mathcal{H}$, no ancestor of leaf $\tilde{s}_i$ satisfies $\mathcal{F}(v) = 0$.

The ZK property of the signature scheme is contingent on Invariant 2 holding for all $i \in \mathcal{H}$. A fault that changes $\mathcal{F}(v)$ from 1 to 0 for any v that is an ancestor of some $\tilde{s}_i$ with $i \in \mathcal{H}$ breaks Invariant 2 and may allow an attacker to reconstruct $\tilde{s}_i$.

3.2 Mapping Schemes to the GZKST Model

We now establish that each of the ZK-based post-quantum signature schemes introduced in Section 2 is a valid GZKST instance in the sense of Definition 2. For each scheme, we exhibit the concrete instantiation of the six algorithms and verify that Invariants 1 and 2 hold by construction.

Proposition 1 (LESS-v2 is a GZKST instance). *LESS-v2 instantiates Definition 2 with secret type $\mathcal{S} = \{\pi_1, \dots, \pi_{s-1}\}$ (the $s - 1$ secret monomial permutations), t parallel repetitions, and an incomplete GGM tree of height $\lceil \log_2 t \rceil$. Invariants 1 and 2 hold by construction of the three-phase flag tree; see Appendix A.1 for the full proof.*

Remark 1 (Invariants under incomplete trees). When the tree is incomplete, some leaves appear at depth $\lceil \log_2 t \rceil - 1$. The index-tracking arrays `npl`, `ll`, `off`, and `lsi` ensure that `LabelLeaves` and `ComputeSeeds` visit the correct nodes regardless of level. Both invariants therefore hold for any tree shape produced by the LESS-v2 expansion (see Appendix A.1).

- `Setup( $1^\lambda$ )` $\rightarrow$ (`seedr`, `salt`): The root seed is derived from the secret key seed; the salt is sampled uniformly at random.
- `Expand(seedr, salt)` $\rightarrow$ ($\mathcal{T}, (\tilde{s}_0, \dots, \tilde{s}_{t-1})$): LESS-v2 uses an incomplete GGM tree with $\lceil \log_2 t \rceil$ levels. Leaves are distributed across the bottom two levels; index offsets are tracked via `npl`, `ll`, `off`, and `lsi` (Figure 2a).
- `Chal(cmt, msg)` $\rightarrow$ `c`: The t canonical-form matrices together with the salt and message are hashed; `SampleChallenge` produces $\mathbf{b} \in \{0, \dots, s - 1\}^t$ of weight w . Setting $c_i = b^{(i)}$, the condition $c_i = 0$ (resp. $c_i \neq 0$) signals a revealed (resp. hidden) round.
- `Decide(c)` $\rightarrow$ ($\mathcal{H}, \mathcal{R}$): $\mathcal{H} = \text{supp}(\mathbf{b})$, $\mathcal{R} = \{0, \dots, t - 1\} \setminus \mathcal{H}$.
- `Publish( $\mathcal{T}, \mathcal{H}$ )` $\rightarrow$ `path`: LESS-v2 builds an auxiliary flag tree of the same shape via two phases: (i) *leaf labelling* (`LabelLeaves`): $\mathcal{F}(\text{leaf}_i) = \mathbf{1}[i \in \mathcal{H}]$; (ii) *upward propagation* (`ComputeSeeds`): $\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r)$ for each internal node v with children ℓ and r . The two-phase description is an implementation

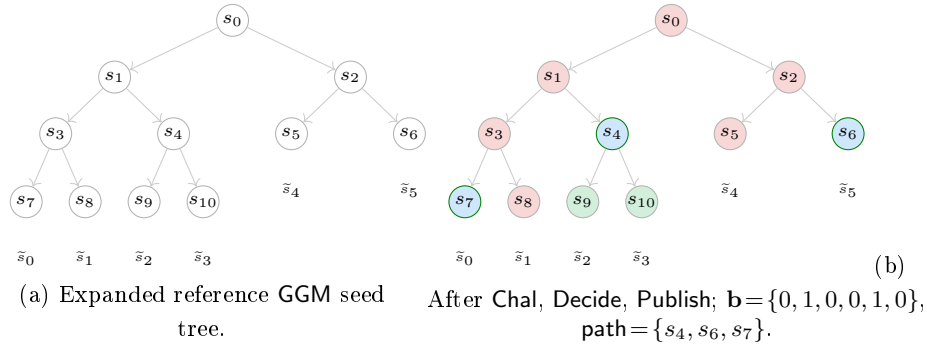


Fig. 2: LESS-v2 GGM seed tree ($t = 6$): structure (a) and flag state (b). In (b): pink nodes ($\mathcal{F}=1$, hidden subtree); green nodes ($\mathcal{F}=0$, fully revealed subtree); blue-bordered nodes ($\mathcal{F}=0$, in path).

303 detail; the resulting flag assignment is exactly the OR-propagation of equations (2)–
 304 (3) (the root’s flag is determined solely by this propagation). The path
 305 $\text{path} = \{v : \mathcal{F}(v) = 0, \mathcal{F}(\text{parent}(v)) = 1\}$ (with the convention that
 306 $\mathcal{F}(\text{parent}(\text{root})) = 1$) is the minimal publishable boundary (Figure 2b).
 307 – $\text{Recon}(\text{path}, \mathbf{c}, \text{salt}) \rightarrow \{\tilde{s}_i : i \in \mathcal{R}\}$: PathToSeedTree places received nodes
 308 on an empty tree of the same shape and re-expands downward via G until
 309 all t leaves are filled. Positions $i \in \mathcal{H}$ remain empty; positions $i \in \mathcal{R}$ yield
 310 the recovered seeds from which monomial matrices and commitments are
 311 re-derived.

312 **Proposition 2 (MEDS is a GZKST instance).** *MEDS instantiates Definition 2*
 313 *with secret type $\mathcal{S} = \{(A_j^{-1}, B_j^{-1})\}_{j=1}^{s-1}$, t parallel repetitions, and a GGM tree*
 314 *of height $\lceil \log_2 t \rceil$ (complete when t is a power of two, incomplete otherwise;*
 315 *only the first t of the $2^{\lceil \log_2 t \rceil}$ leaves are used). Invariants 1 and 2 hold by*
 316 *the SeedTreeToPath construction regardless of whether the tree is complete or*
 317 *incomplete; see Appendix A.2.*

318 – $\text{Setup}(1^\lambda) \rightarrow (\text{seed}_r, \text{salt})$: MEDS samples a fresh random scalar and derives
 319 the tree seed and salt via a single XOF call.
 320 – $\text{Expand}(\text{seed}_r, \text{salt}) \rightarrow (\mathcal{T}, (\tilde{s}_0, \dots, \tilde{s}_{t-1}))$: A binary tree of height $\lceil \log_2 t \rceil$ is
 321 built by hashing each parent with the salt and node index; the first t leaves
 322 are returned. When t is not a power of two, the rightmost $2^{\lceil \log_2 t \rceil} - t$ potential
 323 leaves are unused, making the tree incomplete—the same structural situation
 324 as LESS-v2.
 325 – $\text{Chal}(\text{cmt}, \text{msg}) \rightarrow \mathbf{c}$: Each leaf is expanded into sub-seeds via XOF, ephemeral
 326 matrices are sampled, and the commitment is the hash of all compressed
 327 generator matrices together with m . Parsing the digest via $\text{ParseHash}_{s,t,w}$
 328 yields $\mathbf{h} \in \{0, \dots, s-1\}^t$ of weight w ; setting $c_i = h_i$, $c_i = 0$ (resp. $c_i \neq 0$)
 329 signals a revealed (resp. hidden) round.

- 330 – $\text{Decide}(\mathbf{c}) \rightarrow (\mathcal{H}, \mathcal{R})$: $\mathcal{H} = \{i : c_i \neq 0\}$, $\mathcal{R} = \{i : c_i = 0\}$.
- 331 – $\text{Publish}(\mathcal{T}, \mathcal{H}) \rightarrow \text{path}$: SeedTreeToPath_t removes the label of each leaf $i \in \mathcal{H}$
- 332 and propagates each deletion upward, erasing every ancestor that has at least
- 333 one hidden leaf in its subtree (i.e., the same bottom-up $\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r)$
- 334 rule as LESS-v2). Surviving boundary nodes—those with $\mathcal{F}(v) = 0$ and
- 335 $\mathcal{F}(\text{parent}(v)) = 1$ —are serialised in depth-first order.
- 336 – $\text{Recon}(\text{path}, \mathbf{c}, \text{salt}) \rightarrow \{\tilde{s}_i : i \in \mathcal{R}\}$: PathToSeedTree_t places path nodes on an
- 337 empty tree and re-expands downward via G ; positions $i \in \mathcal{H}$ remain empty.

338 *Remark 2.* In this instantiation of the Publish algorithm, we rely on the path
 339 construction as described in the MEDS submission [7]. However, the reference
 340 implementation uses a different algorithm, that is further explained in Section 4.

341 **Proposition 3 (CROSS is a GZKST instance).** *CROSS instantiates Definition 2*
 342 *with secret type $\mathcal{S} = \{\mathbf{e}\}$ (the R-SDP solution), t parallel repetitions, and a*
 343 *complete GGM tree. The seed-publishing component satisfies Invariants 1 and 2*
 344 *by the same argument as MEDS (Proposition 2). The additional Merkle proof*
 345 *over $\{\text{cmt}_0[i]\}_{i \in \mathcal{H}}$ is a CROSS-specific commitment-authenticity mechanism orthogonal*
 346 *to the GZKST seed-hiding invariants.*

- 347 – $\text{Setup}(1^\lambda) \rightarrow (\text{seed}_r, \text{salt})$: Root seed of size λ bits and salt of size 2λ bits,
- 348 both drawn uniformly at random.
- 349 – $\text{Expand}(\text{seed}_r, \text{salt}) \rightarrow (\mathcal{T}, (\tilde{s}_0, \dots, \tilde{s}_{t-1}))$: A complete binary tree (CROSS-
- 350 v1 identical to LESS-v1; CROSS-v2 uses the SeedTreePath mechanism of
- 351 MEDS) where each parent seed is concatenated with the salt and node index
- 352 and processed by SHAKE to yield two λ -bit child seeds.
- 353 – $\text{Chal}(\text{cmt}, \text{msg}) \rightarrow \mathbf{c}$: CROSS applies Fiat–Shamir in two rounds. The t
- 354 commitment pairs $(\text{cmt}_0[i], \text{cmt}_1[i])$, the salt, and the message are hashed
- 355 to produce $\text{chall}_1 \in (\mathbb{F}_p^*)^t$, from which first-response digests $y_i = \text{Hash}(\mathbf{u}'_i +$
- 356 $\text{chall}_1[i] \cdot \mathbf{e}'_i)$ are derived. A second hash yields $\text{chall}_2 \in \mathcal{B}(t, w)$; setting
- 357 $c_i = \text{chall}_2[i] \in \{0, 1\}$, $c_i = 0$ (resp. 1) signals a revealed (resp. hidden)
- 358 round.
- 359 – $\text{Decide}(\mathbf{c}) \rightarrow (\mathcal{H}, \mathcal{R})$: $\mathcal{H} = \{i : c_i = 1\}$, $\mathcal{R} = \{i : c_i = 0\}$.
- 360 – $\text{Publish}(\mathcal{T}, \mathcal{H}) \rightarrow \text{path}$: The seed path is computed as in MEDS. Additionally,
- 361 a Merkle proof over $\{\text{cmt}_0[i]\}_{i \in \mathcal{H}}$ authenticates the w commitment values
- 362 that the verifier cannot recompute; this is orthogonal to the GZKST seed-
- 363 hiding invariants (see Proposition 3).
- 364 – $\text{Recon}(\text{path}, \mathbf{c}, \text{salt}) \rightarrow \{\tilde{s}_i : i \in \mathcal{R}\}$: Path nodes are placed on an empty tree
- 365 and re-expanded downward. Positions $i \in \mathcal{H}$ remain empty; positions $i \in \mathcal{R}$
- 366 yield seeds from which $(\mathbf{e}'_i, \mathbf{u}'_i)$ are re-derived and $\text{cmt}_1[i]$ is recomputed for
- 367 verification.

368 **Definition 3 (Faulted Signing Oracle).** *Fix a node $v^* \in V(\mathcal{T})$. The faulted*
 369 *signing oracle $\mathcal{O}^{v^*}(\text{sk}, \text{pk})$ takes a message msg and executes the signing algorithm*
 370 *with a single modification: during $\text{Publish}(\mathcal{T}, \mathcal{H})$, the flag value is forced to*

$$\mathcal{F}(v^*) \leftarrow 0,$$

371 regardless of whether $\mathcal{F}(v^*)$ equals 1 in the honest execution. Upward propagation
 372 is applied as normal after this assignment. The oracle returns the resulting
 373 (possibly faulted) signature sig^* .

374 3.3 Oracle Abstraction

375 We previously presented the abstract view of the GGM-tree and its instantiations
 376 in LESS, MEDS, and CROSS. We now formalize the attacker's interface in an
 377 abstract manner, extending the GZKSTtuple with a fault model and a secret-
 378 extraction procedure.

379 *Faulted signing oracle.* Let $\Pi = (\text{Setup}, \text{Expand}, \text{Chal}, \text{Decide}, \text{Publish}, \text{Recon})$ be
 380 a GZKST instance with secret type $\mathcal{S}$, and let (sk, pk) be a fixed key pair. We
 381 model the adversary's access to the victim's signing device as a *faulted oracle*:

382 **Definition 4 (Faulted Signing Oracle).** Fix a node $v^* \in V(\mathcal{T})$. The faulted
 383 signing oracle $\mathcal{O}^{v^*}(\text{sk}, \text{pk})$ takes a message msg and executes the signing algorithm
 384 with a single modification: during $\text{Publish}(\mathcal{T}, \mathcal{H})$, the flag value is forced to

$$\mathcal{F}(v^*) \leftarrow 0,$$

385 regardless of whether $\mathcal{F}(v^*)$ equals 1 in the honest execution. Upward propagation
 386 is applied as normal after this assignment. The oracle returns the resulting
 387 (possibly faulted) signature sig^* .

388 The fault is *effective* if the original value $\mathcal{F}(v^*)$ was 1 and v^* is an ancestor of at
 389 least one leaf $\tilde{s}_i$ with $i \in \mathcal{H}$: in this case, the published path path now contains
 390 a node from which $\tilde{s}_i$ is derivable, directly violating Invariant 2. The fault is
 391 *ineffective* otherwise (the node was already publishable, or all hidden leaves lie
 392 in the sibling subtree).

393 *Remark 3.* The fault model of Definition 4 is realised by several concrete physical
 394 mechanisms: an instruction-skip fault that bypasses the store $\mathcal{F}(v) \leftarrow \mathcal{F}(\ell) \vee$
 395 $\mathcal{F}(r)$, a stuck-at-zero perturbation forcing $\mathcal{F}(v^*) = 0$, a bit-flip ($1 \rightarrow 0$) via
 396 Rowhammer, or a clock-glitch that skips the validity check on a specific tree
 397 node. All of these are documented in the literature [13,14].

398 *Remark 4.* We assume that our faulted oracle has an algorithm `SolverS` that
 399 efficiently solves secrets of type $\mathcal{S}$. For example, when instantiating our oracle
 400 against LESS, we use Lemma 1 from [13] to solve the secret type.

401 *Dual revelation.* The invariant that the Publish phase must enforce is that, for
 402 every hidden round $i \in \mathcal{H}$, the signer simultaneously publishes:

- 403 (i) the ZK response resp_i , which encodes the relationship between $\tilde{s}_i$ and the
- 404 long-term secret sk ; and

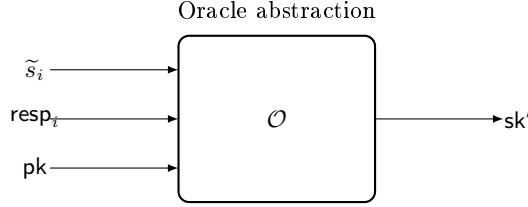


Fig. 3: The **Extract** oracle: given dual revelation $(\tilde{s}_i, \text{resp}_i)$ and public key pk , it outputs partial secret sk' .

405 (ii) a path path from which $\tilde{s}_i$ is *not* derivable.

406 An effective fault breaks condition (ii): the attacker now holds both $\tilde{s}_i$ (reachable
 407 from the faulted path) and resp_i (always present in the signature). This *dual*
 408 *revelation* is the primitive from which all secret-extraction procedures in this
 409 work are built.

410 *Secret extraction.* We extend the GZKST tuple with a seventh, scheme-specific
 411 procedure:

412 **Definition 5 (Extract).** $\text{Extract}(\tilde{s}_i, \text{resp}_i, \text{pk}) \rightarrow \text{sk}'$: given the leaked seed $\tilde{s}_i$
 413 and the ZK response resp_i for the same round $i \in \mathcal{H}$, recover the (partial) secret
 414 $\text{sk}' \subseteq \mathcal{S}$.

415 For the schemes studied in this work, **Extract** takes the following concrete forms:

416 – **MEDS.** For a hidden round i with $h_i = j$, the signer publishes $(\mu_i, \nu_i) =$
 417 $(A_j \tilde{A}_i^{-1}, B_j^{-1} \tilde{B}_i)$ (consistent with the background description in §2.2). Expanding
 418 $\tilde{s}_i$ yields $(\tilde{A}_i, \tilde{B}_i)$, so

$$A_j = \mu_i \cdot \tilde{A}_i, \quad B_j^{-1} = \nu_i \cdot \tilde{B}_i^{-1}.$$

419 Both operations are direct matrix multiplications (and one inversion) over
 420 $\mathbb{F}_q$; no column-reconstruction step is required.

421 – **LESS-v2.** The response is a coset representative $\text{CosetRep}(\pi_{b[i]} \circ \tilde{\pi}_i^*)$. Expanding
 422 $\tilde{s}_i$ gives the partial monomial matrix $\tilde{\pi}_i^*$, and the secret permutation $\pi_{b[i]}$ is
 423 recovered via the column-extraction argument of Lemma 1 in [14].

424 – **CROSS.** The response encodes $\mathbf{v}_i = \mathbf{e} \star (\mathbf{e}'_i)^{-1}$. Expanding $\tilde{s}_i$ gives $\mathbf{e}'_i$, so
 425 $\mathbf{e} = \mathbf{v}_i \star \mathbf{e}'_i$ directly.

426 *Full key recovery.* When $|\mathcal{S}| = s - 1 > 1$ (as in MEDS with $s \geq 3$, and LESS
 427 with $s \geq 3$), a single effective fault recovers only the secret indexed by h_i . Full
 428 recovery requires observing each index $j \in \{1, \dots, s - 1\}$ at least once.

429 **Proposition 4 (Correctness of Algorithm 4).** *Let Π be a GZKST instance,*
 430 *suppose $v^* = \text{root}$ (so the fault exposes all w hidden leaves, making $L = w$*

Algorithm 4: GZKST key recovery from faulted oracle

Input : Faulted oracle $\mathcal{O}^{v^*}$, public key pk
Output : Recovered secret key $\hat{\text{sk}}$

```

1 Collected  $\leftarrow \emptyset$ 
2 while  $|\text{Collected}| < |\mathcal{S}|$  do
3    $\text{sig}^* \leftarrow \mathcal{O}^{v^*}(\text{msg})$  for an arbitrary msg
4   if  $\text{sig}^*$  is effective
5     for each  $i \in \text{leaked}(\text{sig}^*)$  do
6        $\text{sk}' \leftarrow \text{Extract}(\tilde{s}_i, \text{resp}_i, \text{pk})$ 
7        $\text{Collected} \leftarrow \text{Collected} \cup \{(j, \text{sk}')\}$  where  $j = c_i$ 
8 return  $\hat{\text{sk}} \leftarrow \text{Collect}$ 

```

431 deterministic), and let $p_{\min} = 1 - \left(\frac{s-2}{s-1}\right)^w$. Algorithm 4 terminates and outputs
432 $\hat{\text{sk}} = \text{sk}$ with probability at least $1 - \delta$ after at most $\left\lceil \frac{s-1}{p_{\min}} \cdot \ln \frac{1}{\delta} \right\rceil$ effective oracle
433 queries; see Appendix A.4 for the full proof.

434 *Remark 5 (General fault location).* For an effective fault at an arbitrary node v^*
435 with ℓ leaf descendants, the number $L = |\mathcal{H} \cap \text{leaves}(v^*)|$ of hidden leaves in the
436 subtree is a hypergeometric random variable whose realisation may be smaller
437 than its mean $\bar{w} = w\ell/t$, making the per-query success probability $p_r(L) =$
438 $1 - ((s-1-r)/(s-1))^L$ potentially smaller than the root-fault $p_{\min}$. A valid bound
439 for any effective fault is obtained by taking the worst case $L = 1$, which gives
440 $p_r \geq r/(s-1)$ for all $r \geq 1$ and therefore $\mathbb{E}[Q] \leq (s-1)H_{s-1}$, where $H_{s-1} =$
441 $\sum_{r=1}^{s-1} 1/r$ is the $(s-1)$ -th harmonic number. For $s \leq 6$ (all schemes in Table 1),
442 this gives $\mathbb{E}[Q] \leq 5H_5 \approx 11.4$, still a small constant.

443 Detecting whether a received signature is effective is feasible for the adversary:
444 given the fault location v^* and the challenge $\mathbf{c}$ (recoverable from the digest d in
445 the signature), the adversary recomputes the honest flag tree, checks whether
446 $\mathcal{F}(v^*) = 1$, and verifies that at least one hidden leaf falls in the subtree of v^* .
447 This detection is detailed for LESS in [13] and applies unchanged to MEDS and
448 CROSS.

449 *Expected query count.* Let ℓ denote the number of leaves in the subtree rooted
450 at v^* .

451 **Proposition 5.** Let $v^* = \text{root}$, so $L = w$ exactly. Define $p_{\min} = 1 - \left(\frac{s-2}{s-1}\right)^w$. The
452 per-query success probability satisfies $p_r \geq p_{\min}$ for all $r \geq 1$, and the expected
453 total number of effective queries satisfies

$$\mathbb{E}[Q] \leq \frac{s-1}{p_{\min}}.$$

454 See Appendix A.5 for the derivation.

Remark 6 (Role of E_{dist} and Jensen’s inequality). The quantity $E_{\text{dist}} = (s - 1)p_{\min}$ equals the expected number of distinct secret indices revealed per effective query (when $L = w$ is deterministic at the root). When $v^* \neq \text{root}$ and L is random with mean $\bar{w} = w\ell/t$, Jensen’s inequality (the function α^x is convex in x) gives $\mathbb{E}[\# \text{distinct}] \leq E_{\text{dist}}$: E_{dist} is an *upper* bound on expected per-query gain, which implies a *lower* bound on $\mathbb{E}[Q]$, not an upper bound. The upper bound $\mathbb{E}[Q] \leq (s - 1)/p_{\min}$ stated in Proposition 5 follows from the direct inequality $p_r \geq p_{\min}$, not from the Jensen argument.

For all MEDS parameter sets (Table 1), faulting the root ($v^* = \text{root}$, $\ell = t$) gives $\bar{w} = w$; for those parameters $w \gg 1$, so $p_{\min} \approx 1$ and $\mathbb{E}[Q] \lesssim s - 1$, which is at most 5 across all sets.

Remark 7 (Concrete query counts for MEDS). Table 2 reports $p_{\min}$ and $\mathbb{E}[Q]$ for every MEDS parameter set when $v^* = \text{root}$ (so $L = w$ exactly). The values use the exact formula $p_{\min} = 1 - ((s - 2)/(s - 1))^w$; for all parameter sets $p_{\min} > 0.99$, giving $\mathbb{E}[Q] \leq s - 1 \leq 5$.

Table 2: Expected effective queries $\mathbb{E}[Q]$ for MEDS (fault at root).

Parameter set	s	w	$s-1$	$p_{\min}$	$\mathbb{E}[Q] \leq$
MEDS-9923	4	14	3	0.9966	3.01
MEDS-13220	5	20	4	0.9968	4.01
MEDS-41711	4	26	3	1.0000	3.00
MEDS-69497	5	36	4	1.0000	4.00
MEDS-134180	5	52	4	1.0000	4.00
MEDS-167717	6	66	5	1.0000	5.00

From key recovery to forgery. Once Algorithm 4 terminates, the adversary holds a complete signing key sk and can produce valid signatures on any message msg^* — including messages never submitted to any oracle. This constitutes a full break of EUF-CMA:

Corollary 1. *Let Π be a GZKST-based signature scheme and let the fault model be as in Definition 4 (a stuck-at-zero perturbation of a single flag bit during Publish). If an efficient adversary can induce this fault at a fixed node $v^* \in V(\mathcal{T})$ on the victim’s signing device, then Π is not EUF-CMA-secure in the fault model: the adversary recovers sk via Algorithm 4 and forges signatures on arbitrary messages without further oracle access.*

480 4 Fault Injection attacks on MEDS GGM tree procedures

481 This subsection details a fault injection (FI) attack conducted on the C reference
 482 implementation of the MEDS signature algorithm. We focus on exploiting the
 483 tree traversal logic used in the path construction to leak hidden seeds.

484 4.1 Target Analysis

485 The primary target of our analysis is the `SeedTreePath` function, specifically
 486 targeting the logic represented in the signing phase of the MEDS algorithm
 487 (instruction 16 of algorithm 2). In the reference implementation, both `SeedTreePath`
 488 (path generation) and `SeedTree` (tree reconstruction) are handled by a unified
 489 function, `stree_to_path_to_stree`, which operates in two modes defined by a
 490 mode parameter.

Listing 1.1: `stree_to_path_to_stree` implementation.

```

491 1 void stree_to_path_to_stree(uint8_t *stree, uint8_t *h_digest, uint8_t *path, uint8_t *salt
492 2 , int mode) {
493 3     int h = 0, i = 0;
494 4     unsigned int id = 0, idx = 0;
495 5     unsigned int indices[MEDS_w] = {0};
496 6     for (int i = 0; i < MEDS_t; i++) {
497 7         if (h_digest[i] > 0) {
498 8             indices[idx++] = i;
499 9         }
500 10    }
501 11    while (1==1) {
502 12        int index_leaf = 0;
503 13        while ((indices[id] > (MEDS_seed_tree_height-h)) == i)
504 14            { // Traverse down toward the secret leaf
505 15                h += 1;
506 16                i <<= 1;
507 17                if (h > MEDS_seed_tree_height) {
508 18                    h -= 1;
509 19                    i >>= 1;
510 20                    if (id + 1 < MEDS_w) id++;
511 21                    index_leaf = 1; // Flag node as hidden
512 22                    break;
513 23                }
514 24            }
515 25            if (index_leaf == 0) {
516 26                if (mode == STREE_TO_PATH) {
517 27                    memcpy(path, &stree[MEDS_st_seed_bytes * SEED_TREE_ADDR(h, i)],
518 28                        MEDS_st_seed_bytes);
519 29                    path += MEDS_st_seed_bytes;
520 30                } else {
521 31                    memcpy(&stree[MEDS_st_seed_bytes * SEED_TREE_ADDR(h, i)], path,
522 32                        MEDS_st_seed_bytes);
523 33                    path += MEDS_st_seed_bytes;
524 34                    t_hash(stree, salt, h, i);
525 35                }
526 36            }
527 37            // Backtracking logic
528 38            while ((i & 1) == 1) {
529 39                h -= 1;
530 40                i >>= 1;
531 41            }
532 42            i += 1;
533 43            if ((i << (MEDS_seed_tree_height - h)) >= MEDS_t) return;
534 44        }

```

```

535 41 }
536 42 }

```

538 The algorithm identifies secret leaves (where $h[i] > 0$) and performs a Depth-
 539 First Search (DFS). The exploration halts as soon as it reaches a node that is
 540 no longer an ancestor of the hidden leaf. This node is then appended to the
 541 signature path. If the exploration reaches a target hidden leaf, the `index_leaf`
 542 flag is set to ensure the leaf seed remains unpublished and we move on to the
 543 next hidden leaf.

544 **Objective** The adversary’s goal is to manipulate the control flow via fault
 545 injection to leak unpublished seeds (internal nodes or leaves) into the public
 546 signature path, ultimately compromising the security of the signature scheme.

547 4.2 Experimental Setup

548 Our experimental platform consists of a **CW303-STM32F3** target board, featuring
 549 an ARM Cortex-M4 STM32F303RCT7 processor. Clock glitching is performed
 550 using a **ChipWhisperer-Lite**. As a proof-of-concept, the target board is flashed
 551 exclusively with the `stree_to_path_to_stree` function, using fixed values for
 552 the reference tree, salt, and challenge.

553 4.3 Fault Models

554 We propose three fault models targeting different stages of the tree traversal.

555 **Model 1: Full Tree Recovery (Root Leakage)** This attack aims to reveal
 556 the root seed, which effectively compromises the entire tree. By injecting a fault
 557 that causes the skip of the first iteration of `while loop` at line 13 of listing 1.1,
 558 the DFS halts immediately at the root ($h = 0, i = 0$). The root is incorrectly
 559 identified as a node ready for publication.

560 *Detection and recovery.* The signature path contains only a single non-zero
 561 element. Verification is performed by expanding this single seed into a full tree
 562 and checking if the resulting signature is accepted.

563 **Model 2: Subtree Recovery (Index Corruption)** By glitching the loop
 564 `if (id + 1 < MEDS_w) id++;` at line 19 of listing 1.1, the algorithm fails
 565 to update the target index `id`. The traversal logic becomes desynchronized,
 566 resulting in the algorithm treating subsequent branches as public. All leaves
 567 to the right of the faulted `id` become computable.

568 *Detection and recovery.* This fault is hard to detect without a reference signature.
 569 However, one can simulate tree reconstructions for all possible faulted `id` values
 570 to verify which leads to a valid signature. (up to w verifications)

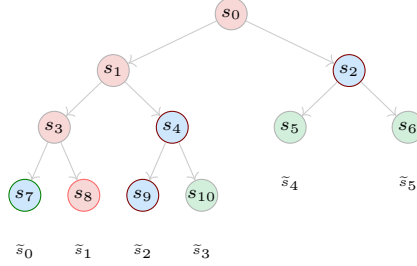


Fig. 4: Effect of fault model 2 on the example presented in figure 2, if the glitch occurred while `id=1`. The returned path is $\{s_7, s_9, s_4, s_2\}$ and all leaves are computable except x_1 .

571 **Model 3: Hidden Leaf Recovery** This model targets the specific instruction
 572 `index_leaf = 1;` at line 20 of listing 1.1. By skipping this assignment, a hidden
 573 leaf is no longer marked as private. The execution continues normally, but the
 574 secret leaf seed is copied into the `path` buffer.

575 *Detection and recovery.* The path length will be $w + k$ (where $k > 0$ is the
 576 number of iterations of the target instruction that were skipped). Detection
 577 involves testing subsets of the path to find a valid signature, which may be
 578 computationally expensive ($\binom{w+k}{w}$ verifications).

579 4.4 Results and Observations

580 We successfully executed the fault models on the Chipwhisperer board. This
 581 section detailed the compiler and glitcher configurations that were required to
 582 obtain the desired faults.

583 **Methodology** For each target fault model, we aim to identify the parameter
 584 configurations that yield the highest attack success rate. First, we define a target
 585 output state that signifies a successful fault injection.

586 Next, we establish a precise faulting window encompassing one or more target
 587 assembly instructions. To synchronize the injection hardware, we instrument
 588 the firmware by inserting a `trigger_high()` instruction immediately before the
 589 target instruction block to signal the ChipWhisperer to begin counting clock
 590 cycles, followed by a `trigger_low()` instruction to terminate the window. We

591 then verify the disassembled binary to ensure that compiler optimizations have
 592 not rearranged, omitted, or reordered these trigger placements.

593 To configure the temporal bounds of the injection, we define the **repeat** parameter,
 594 which specifies the continuous duration (in clock cycles) of the clock glitch.
 595 We calibrate this maximum value based on the nominal cycle execution counts
 596 provided in the ARM documentation for the target instruction sequence.

597 Finally, we iteratively sweep the following spatial and temporal parameters to
 598 map the physical fault space and isolate high-yield vulnerability regions:

- 599 – **width**: The width of the glitch pulse, achieved by XORing the system clock
 600 (expressed as a percentage of the clock period, spanning from 0% to 50%).
- 601 – **offset**: The phase shift or delay introduced before the rising edge of the
 602 glitch pulse relative to the clock cycle (expressed as a percentage of the
 603 clock period, spanning from -50% to 50%).
- 604 – **ext_offset**: The absolute number of clock cycles elapsed after the trigger
 605 signal before the glitch injection begins.

606 **Full tree recovery** This attack is successful if the returned path contains a
 607 single node which is the root node.

608 *Compilation.* This attack is carried out on a code compiled with the default **-Os**
 609 optimization.

610 *Trigger position.* We directly target the evaluation of the conditional and the
 611 branching instruction corresponding to line 19 of 1.1). The compiled result is in
 612 1.2. We fault for 3 clock cycles.

Listing 1.2: Compiled trigger points for fault model 1.

```

613 0800062c <stree_to_path_to_stree_glitch>:
614 ...
615 // trigger_high();
616 8000710: f001 fa12 bl 8001b38 <trigger_high>
617 // while ((indices[id] >> (MEDS_seed_tree_height - h)) == i)
618 8000714: ab06 add r3, sp, #24
619 8000716: eb03 0488 add.w r4, r3, r8, lsl #2
620 800071a: e00d b.n 8000738 <stree_to_path_to_stree_glitch+0x66>
621 // trigger_low();
622 800071c: f001 fa13 bl 8001b46 <trigger_low>
623 ...
624 
```

626 *Parameters.* Figure 5 shows that the attack has a high success rate and lower
 627 reset rates for an **ext_offset** ranging from 1.5 to 2.5, a small **offset** of -1 to
 628 1 and rather wide glitches (**width** over 20).

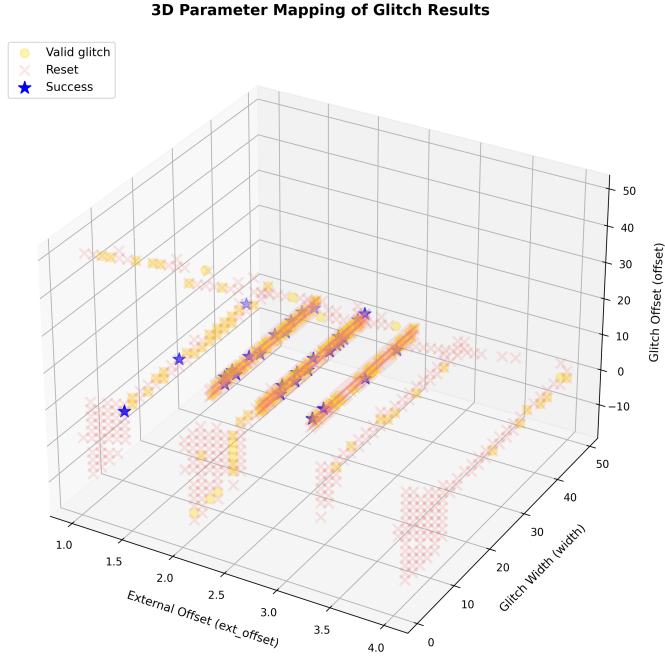


Fig. 5: Glitch success frequency by glitch width and offset for fault model 1, with 10 tries per parameter set. Valid glitches are runs that did not crash and output paths that were neither correct nor the target faulted path.

629 Subtree recovery

630 *Compilation.* This is carried out on a code compiled with the default `-Os`
 631 optimization.

632 *Trigger position.* We set the trigger to frame lines 19 of 1.1. The compiled result
 633 is in 1.3.

Listing 1.3: Compiled trigger points for fault model 2.

```

634 080006d2 <stree_to_path_to_stree_glitch>:
635 ...
636 ...
637 8000726: f001 fa09 b1 8001b3c <trigger_high> // trigger_high();
638 800072a: f108 0401 add.w r4, r8, #1
639 800072e: 2c05 cmp r4, #5 //if (id+1 < MEDS_w)
640 8000730: bf88 it hi
641 8000732: 4644 movhi r4, r8 // id++ ;
642 8000734: f001 fa09 b1 8001b4a <trigger_low> // trigger_low();
643 ...

```

645 *Parameters.* The evaluation of the condition and the incrementation of `id`
 646 require 3 clock cycles. We therefore set `repeat` to 3. We consider the attack
 647 successful if we obtain the faulted path pattern described by figure 4.

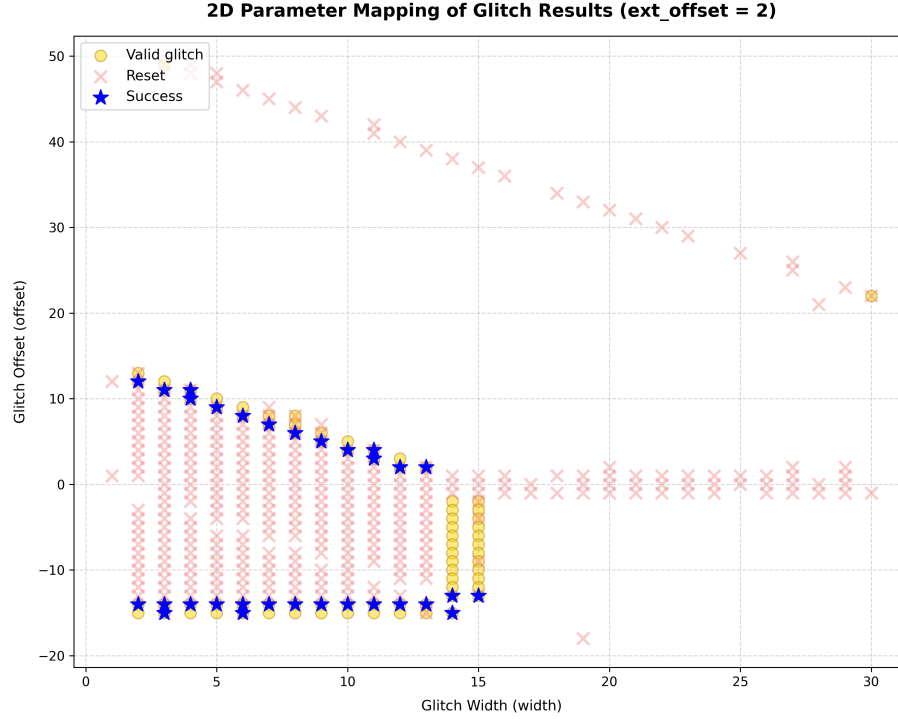


Fig. 6: Glitch success frequency by glitch width and offset for fault model 2, with 10 tries per parameter pair.

648 We observe a distinct, bounded region of system resets delimited by successful
 649 attacks and other valid glitches. Crucially, the paths of many of these valid
 650 glitches contain a large set of intermediate nodes which provide sufficient coverage
 651 to compute all of the leaf values.

652 Single hidden leaf recovery

653 *Compilation.* Higher compiler optimization levels made it difficult to precisely
 654 target the flags. Specifically, the `-O1`, `-O2`, `-O3`, and `-Os` optimization flags
 655 resulted in compiled assembly code that omitted the explicit assignment instruction
 656 entirely, relying instead on complex control-flow manipulations to achieve the
 657 same logical outcome. To resolve this issue, we enforced the `-O0` optimization

level strictly on the targeted function. Applying -O0 to the entire project was unfeasible, as it increased the binary size beyond the storage capacity of the target board.

The disassembled code shows that the assignment is done in two instructions, first The CPU loads the immediate integer value 1 directly into the register r3, then it takes that value 1 from r3 and writes it out to memory.

Trigger position. For the unoptimized version, we position the glitch high trigger right before the `index_leaf` assignment and the low trigger right after. This results in the compiled code in listing 1.4.

Listing 1.4: Compiled trigger points for fault model 3.

```

667 0800062c <stree_to_path_to_stree_glitch>:
668 ...
669 80006be: f001 fd3f bl 8002140 <trigger_high> // trigger_high();
670 80006c2: 2301 movs r3, #1 // index_leaf = 1;
671 80006c4: 62bb str r3, [r7, #40] @ 0x28
672 80006c6: f001 fd43 bl 8002150 <trigger_low> // trigger_low();
673 ...
674

```

Parameters. According to ARM documentation, each of these instruction is 1 clock cycle long. We set the **repeat** parameter of the glitch controller to 2.

Results. The experiments confirm that the attack is reproducible. Across all parameter sweeps, attempts strictly resulted in either normal execution, a device reset, or the expected exploit. We observed a complete absence of other valid glitches. This is due to our single-instruction targeting. Unlike fault models that disrupt larger execution windows and introduce non-deterministic errors, this injection is less likely to bleed into surrounding cycles or to trigger a control-flow collapse.

These experiments demonstrate that the root reveal fault model, which is the most efficient approach requiring the fewest queries for full secret key recovery, is highly reproducible. Furthermore, our results confirm the viability of two alternative fault models, although they prove more challenging to replicate under experimental conditions. Despite their lower reproducibility rates, fault models 2 and 3 remain relevant for threat modeling; as detailed in Section 5, they incur a higher computational or architectural cost to detect during signature generation.

5 Countermeasures

In this section we suggest a set of countermeasures that would make the signature scheme more secure against the exposed fault models.

We split the countermeasures into different categories :

- Post-publication code path check : same signature size, no modification of the tree construction.

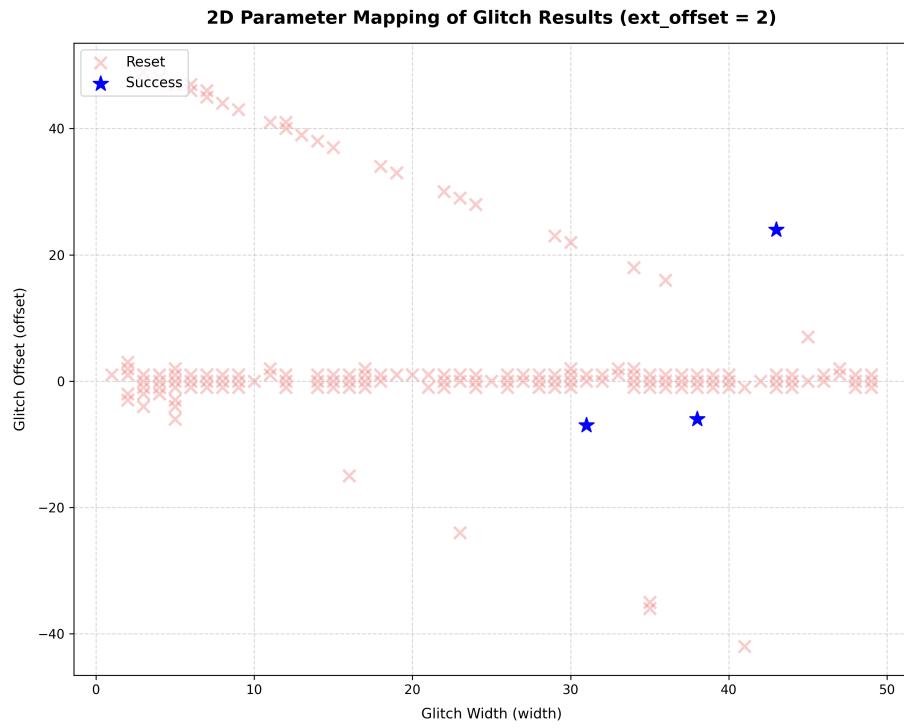


Fig. 7: Glitch success frequency by glitch width and offset for fault model 3, with 10 tries per parameter pair.

- 698 – Path construction algorithm change : same signature size.
- 699 – GGM tree-less signature : larger signature.

700 5.1 Post-publication path check

701 This countermeasure consists is running checks on the output of `SeedTreeToPath`.

702 A simple size check on the path is enough to be secure against fault models 1
 703 and 3, since they result in path lengths of 1 and golden path size+1 respectively.
 704 The expected path size can be computed in linear time from $\mathcal{H}$. A more thorough
 705 check would be to rebuild the tree using `PathToSeedTree` on the obtained path
 706 and to compare the obtained leaves to $\mathcal{H}$ and the reference tree.

707 *Remark 8.* In order to pass this check, it would be sufficient to induce the same
 708 fault as in the `SeedTreeToPath` on `PathToSeedTree`, seeing as they both call the
 709 same function in the reference implementation.

710 5.2 Path construction algorithm modification

711 Although the unified implementation of `SeedTreeToPath` and `PathToSeedTree`
 712 is highly efficient, its structural coupling of the flag tree construction and path
 713 extraction phases creates a vulnerability. Decoupling these operations into independent
 714 processing stages, and introducing validation checks after each phase, reduces
 715 the attack surface. This structural redundancy ensures that any fault injected
 716 during the initial flag tree generation will cause a detectable mismatch during
 717 the subsequent path construction phase.

718 We implement this countermeasure using the following multi-stage verification
 719 procedure:

- 720 1. Construct a binary flag tree $\mathcal{F}$ derived from $\mathcal{H}$.
- 721 2. Verify that for every leaf position i where $\mathcal{F}[i] = 1$, no ancestral node in its
 722 path to the root is flagged as 0.
- 723 3. Reconstruct the node path using the validated flag tree $\mathcal{F}$ and the reference
 724 seed tree.
- 725 4. Assert that no leaf node explicitly flagged as 1 is included in the final
 726 extracted path.

727 *Complexity.* All four verification steps scale linearly with the size of the tree,
 728 maintaining a time complexity of $\mathcal{O}(t)$.

729 Impact on performances

730 *TODO* add third countermeasure + on other architectures.

731 We implement both of these checks and compare execution times of the signature
 732 step with or without countermeasure. The results are summarized in table 3. We
 733 refer to the path size check countermeasure by C_{size} and the full tree check by
 734 C_{tree} .

Table 3: Countermeasures execution time comparison

Version	Time (min)	cycles
Base	0.001230	4798061
C_{size}	0.001070	4171382
C_{tree}	0.001303	5080762

735 5.3 GGM-tree-less signature

736 Alternative architectural designs could consider abandoning GGM tree structures
 737 entirely. A potential solution, adapting the fault attack countermeasure for LESS
 738 proposed by [13] would require the response vector v to explicitly contain the
 739 precomputed matrices $\tilde{A}_i$ and $\tilde{B}_i$ that are sampled from the published leaves σ_i .

$$v[i] = \begin{cases} (\tilde{A}_i.A_{h_i}^{-1}, \tilde{B}_i.B_{h_i}^{-1}), & \text{if } h_i > 0 \\ (\tilde{A}_i, \tilde{B}_i), & \text{if } h_i = 0 \end{cases} \quad (1)$$

740 While this design ensures that the secret matrices $(\tilde{A}_i, \tilde{B}_i)$ remain computationally
 741 infeasible to derive for any index i , it introduces a disadvantage by significantly
 742 inflating the overall signature size.

743 *TODO.* Signature size comparison by parameter.

744 References

- 745 1. Emmanuelle Anceaume, Yann Busnel, Ernst Schulte-Geers, and Bruno Sericola.
 746 Optimization results for a generalized coupon collector problem. *J. Appl. Probab.*,
 747 53(2):622–629, 2016.
- 748 2. M. Baldi, A. Barengi, L. Beckwith, J.-F. Biasse, T. Chou, A. Esser,
 749 K. Gaj, P. Karl, K. Mohajerani, G. Pelosi, E. Persichetti, M.-J. O.
 750 Saarinen, P. Santini, R. Wallace, and F. Zveydinger. LESS: Linear code
 751 equivalence signature scheme — round 2 specification. NIST PQC Round 2,
 752 2025. [https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/](https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/less-spec-round2-web.pdf)
 753 [round-2/spec-files/less-spec-round2-web.pdf](https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/less-spec-round2-web.pdf).
- 754 3. Marco Baldi, Alessandro Barengi, Michele Battagliola, Sebastian Bitzer, Marco
 755 Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi,
 756 Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia

- 757 Wachter-Zeh, and Violetta Weger. CROSS: Codes and restricted objects signature
758 scheme — specification document, version 2.0. NIST PQC Round 2 submission,
759 2025.
- 760 4. Marco Baldi, Sebastian Bitzer, Alessio Pavoni, Paolo Santini, Antonia Wachter-
761 Zeh, and Violetta Weger. Zero knowledge protocols and signatures from the
762 restricted syndrome decoding problem. In *Public-Key Cryptography — PKC 2024*,
763 *Part II*, volume 14603 of *Lecture Notes in Computer Science*, pages 243–274.
764 Springer, 2024.
- 765 5. Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem,
766 Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini,
767 Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl.
768 FAEST v2: Algorithm Specifications. [https://faest.info/faest-spec-v2.0.](https://faest.info/faest-spec-v2.0.pdf)
769 [pdf](https://faest.info/faest-spec-v2.0.pdf), February 2025. Accessed: 2026-05-13.
- 770 6. Loïc Bidoux, Jesús-Javier Chi-Domínguez, Thibault Feneuil, Philippe Gaborit,
771 Antoine Joux, Matthieu Rivain, and Adrien Vinçotte. RYDE: a digital signature
772 scheme based on rank syndrome decoding problem with MPC-in-the-Head
773 paradigm. *Designs, Codes and Cryptography*, 93(5):1451–1486, May 2025.
- 774 7. Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Lars Ran,
775 Tovahery Hajatiana Randrianarisoa, Krijn Reijnders, Simona Samardjiska,
776 and Monika Trimoska. MEDS post-quantum cryptography. [https:](https://www.meds-pqc.org/)
777 [//www.meds-pqc.org/](https://www.meds-pqc.org/), 2023.
- 778 8. Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Tovahery Hajatiana
779 Randrianarisoa, Krijn Reijnders, Simona Samardjiska, and Monika Trimoska.
780 Take your MEDS: digital signatures from matrix code equivalence. In Nadia El
781 Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology -*
782 *AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa*,
783 *Sousse, Tunisia, July 19-21, 2023, Proceedings*, Lecture Notes in Computer Science,
784 pages 28–52. Springer, 2023.
- 785 9. Özgür Dagdelen, Marc Fischlin, and Tommaso Gagliardoni. The fiat-shamir
786 transformation in a quantum world. In Kazue Sako and Palash Sarkar, editors,
787 *Advances in Cryptology - ASIACRYPT 2013 - 19th International Conference on*
788 *the Theory and Application of Cryptology and Information Security, Bengaluru,*
789 *India, December 1-5, 2013, Proceedings, Part II*, Lecture Notes in Computer
790 Science, pages 62–81. Springer, 2013.
- 791 10. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to
792 identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in*
793 *Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*,
794 Lecture Notes in Computer Science, pages 186–194. Springer, 1986.
- 795 11. O. Goldreich, S. Goldwasser, and S. Micali. How to construct random functions.
796 *Journal of the ACM*, 33(4):792–807, 1986.
- 797 12. Xiao Huang, Zhuo Huang, Yituo He, Quan Yuan, Chao Sun, Mehdi Tibouchi, and
798 Yu Yu. Faultless key recovery: Iteration-skip and loop-abort fault attacks on LESS.
799 *IACR Cryptol. ePrint Arch.*, 2026:117, 2026.
- 800 13. Puja Mondal, Supriya Adhikary, Suparna Kundu, and Angshuman Karmakar.
801 Zkfault: Fault attack analysis on zero-knowledge based post-quantum digital
802 signature schemes. In Kai-Min Chung and Yu Sasaki, editors, *Advances in*
803 *Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory*
804 *and Application of Cryptology and Information Security, Kolkata, India, December*
805 *9-13, 2024, Proceedings, Part VIII*, Lecture Notes in Computer Science, pages 132–
806 167. Springer, 2024.

14. Puja Mondal, Suparna Kundu, Hikaru Nishiyama, Supriya Adhikary, Daisuke Fujimoto, Yuichi Hayashi, and Angshuman Karmakar. Fault to forge: Fault assisted forging attacks on LESS signature scheme. *IACR Cryptol. ePrint Arch.*, 2025:1838, 2025.
15. Weiyu Xu and Ao Kevin Tang. A generalized coupon collector problem. *CoRR*, abs/1010.5608, 2010.

A Full Proofs

Throughout this appendix, $\mathcal{T}$ denotes a GGM seed tree with leaf set $\{0, \dots, t-1\}$ and $\mathcal{F} : V(\mathcal{T}) \rightarrow \{0, 1\}$ its flag labelling, as produced by the Publish phase of the relevant GZKST instance. We use the convention $\mathcal{F}(v) = 1$ to mean that v 's subtree contains at least one hidden leaf ($i \in \mathcal{H}$), and $\mathcal{F}(v) = 0$ to mean that all leaf descendants of v are revealed ($i \in \mathcal{R}$).

A.1 Proof of Proposition 1 (LESS-v2 is a GZKST instance)

Let $\mathcal{T}$ be the LESS-v2 seed tree with $\lceil \log_2 t \rceil$ levels. After the three-phase flag-tree construction the flag values are:

$$\mathcal{F}(\text{leaf}_i) = \mathbf{1}[i \in \mathcal{H}], \quad i \in \{0, \dots, t-1\}, \quad (2)$$

$$\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r), \quad \text{for every internal node } v \text{ with children } \ell, r. \quad (3)$$

Equations (2)–(3) jointly imply:

$$\mathcal{F}(v) = 1 \iff \exists i \in \mathcal{H} : \text{leaf}_i \text{ is a descendant of } v. \quad (4)$$

The published path is $\text{path} = \{v : \mathcal{F}(v) = 0, \mathcal{F}(\text{parent}(v)) = 1\}$ (with the convention that $\mathcal{F}(\text{parent}(\text{root})) = 1$).

Proof (Proof of Invariant 1 (Correctness)). Let $i \in \mathcal{R}$. Consider the root-to-leaf path $\pi_i = (v_0 = \text{root}, v_1, \dots, v_d = \text{leaf}_i)$.

Since $|\mathcal{H}| \geq 1$ (the challenge weight satisfies $w \geq 1$), at least one leaf has flag 1. By (4), $\mathcal{F}(v_0) = \mathcal{F}(\text{root}) = 1$.

Since $i \in \mathcal{R}$, equation (2) gives $\mathcal{F}(v_d) = 0$.

Because $\mathcal{F}(v_0) = 1$ and $\mathcal{F}(v_d) = 0$, the path π_i contains at least two nodes with different flags. Define $k = \min\{j \in \{0, \dots, d\} : \mathcal{F}(v_j) = 0\}$. Then $k \geq 1$ (since $\mathcal{F}(v_0) = 1$), $\mathcal{F}(v_k) = 0$, and $\mathcal{F}(v_{k-1}) = 1$. Hence $v_k \in \text{path}$, and v_k lies on the root-to-leaf path π_i , establishing Invariant 1.

Proof (Proof of Invariant 2 (ZK hiding)). Let $i \in \mathcal{H}$ and let v be any proper ancestor of leaf_i (including the root). Since leaf_i is a descendant of v and $i \in \mathcal{H}$, equation (4) gives $\mathcal{F}(v) = 1$. Hence no ancestor of leaf_i can satisfy $\mathcal{F}(v) = 0$, so no ancestor of leaf_i lies in path . Invariant 2 holds.

Proof (Extension to incomplete trees (Remark 1)). When t is not a power of 2, the LESS-v2 tree is incomplete: some leaves appear at depth $\lceil \log_2 t \rceil - 1$ rather than $\lceil \log_2 t \rceil$. The index-tracking arrays `npl`, `ll`, `off`, and `lsi` ensure that `LabelLeaves` correctly identifies which nodes are leaves and assigns flags according to (2), and that `ComputeSeeds` performs the bottom-up OR pass (3) over the actual tree structure.

The proofs of both invariants above are purely graph-theoretic and depend only on the properties (2)–(4), which hold regardless of whether leaves appear at a uniform depth. Hence both invariants hold for any incomplete tree shape produced by the LESS-v2 expansion.

A.2 Proof of Proposition 2 (MEDS is a GZKST instance)

`SeedTreeToPath` implements the following erasure: for each $i \in \mathcal{H}$ the algorithm (i) removes the label of `leafi`, and (ii) propagates the removal upward—a node v loses its label as soon as *at least one* leaf in its subtree has been removed. By induction this is equivalent to the OR-propagation $\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r)$ of LESS-v2.

Claim. After the erasure pass, a node v retains its label if and only if $\mathcal{F}(v) = 0$ in the sense of (4), i.e., *every* leaf descendant of v is in $\mathcal{R}$ (no hidden leaf in v ’s subtree).

Proof. ($\Rightarrow$) Suppose v is retained. By definition of the upward propagation, no ancestor of any hidden leaf is retained. Hence v ’s subtree contains no hidden leaf, i.e., $\mathcal{F}(v) = 0$.

($\Leftarrow$) Suppose $\mathcal{F}(v) = 0$, i.e., every leaf in v ’s subtree is in $\mathcal{R}$. The erasure step removes only hidden leaves and their ancestors. Since v ’s subtree contains no hidden leaf, no removal is triggered in v ’s subtree, so v retains its label.

The published path `path` consists of the highest surviving nodes in the tree, serialised in depth-first order. Equivalently,

$$\text{path} = \{v : v \text{ retains its label and } \text{parent}(v) \text{ was erased}\}.$$

By the claim, this is $\{v : \mathcal{F}(v) = 0, \mathcal{F}(\text{parent}(v)) = 1\}$, which is identical to the LESS-v2 definition.

Invariants 1 and 2 follow by the same argument as in Appendix A.1.

A.3 Proof of Proposition 3 (CROSS is a GZKST instance)

The seed-tree component of CROSS (`SeedTreePath`) is structurally equivalent to MEDS’s `SeedTreeToPath` (see Section 3.2). The argument of Appendix A.2 therefore applies directly, establishing Invariants 1 and 2 for the seed-hiding part.

872 The additional Merkle proof over $\{\text{cmt}_0[i]\}_{i \in \mathcal{H}}$ is an authenticity mechanism for
 873 the commitment digests that the verifier cannot recompute from the revealed
 874 seeds. It neither publishes any seed in $\{\tilde{s}_i : i \in \mathcal{H}\}$ nor affects the seed-path
 875 computation, and is therefore orthogonal to both invariants.

876 A.4 Proof of Proposition 4 (Correctness of Algorithm 4)

877 Let $m = |\mathcal{S}| = s - 1$. The process of recovering the secret components is modeled
 878 as a *generalized coupon-collector problem* with group drawings [15,1], where each
 879 query draws a batch of w coupons out of m distinct types.

880 *Correctness of Extract.* For each scheme, Definition 5 specifies **Extract** as a
 881 closed-form algebraic procedure: matrix inversion over $\mathbb{F}_q$ (MEDS), column extraction
 882 (LESS), or a group operation (CROSS). Each procedure is deterministic and
 883 correct given the dual revelation $(\tilde{s}_i, \text{resp}_i)$: the algebraic relationship between
 884 the response and the hidden seed is an identity (not an approximation), so **Extract**
 885 returns exactly $\text{sk}' =$ the secret component indexed by $j = c_i$.

886 We work under the assumption of Proposition 4: $v^* = \text{root}$, so the subtree of v^*
 887 is the entire tree and $L = |\mathcal{H} \cap \text{leaves}(\text{root})| = |\mathcal{H}| = w$ exactly—a deterministic
 888 quantity, not a random variable. Define

$$p_{\min} = 1 - \left(\frac{s-2}{s-1}\right)^w = 1 - \left(1 - \frac{1}{s-1}\right)^w.$$

889 For any $r \geq 1$ uncollected secrets, the probability that at least one of the w
 890 hidden leaves reveals a new secret index is

$$p_r = 1 - \left(1 - \frac{r}{s-1}\right)^w \geq 1 - \left(1 - \frac{1}{s-1}\right)^w = p_{\min} > 0,$$

891 where the inequality holds because $1 - r/(s-1)$ is decreasing in r , so $r = 1$ gives
 892 the weakest bound. Hence

$$\Pr[\text{new}(q) \geq 1 \mid \text{Collected}] \geq p_{\min} > 0 \quad \text{for all } s \geq 2, w \geq 1.$$

893 Let T_j be the number of effective queries to go from $|\text{Collected}| = j - 1$ to
 894 $|\text{Collected}| = j$. Conditional on $|\text{Collected}| = j - 1$, each query succeeds with
 895 probability at least $p_{\min}$, so T_j is stochastically dominated by a $\text{Geom}(p_{\min})$
 896 random variable, which is finite almost surely. Hence $Q = \sum_{j=1}^m T_j$ is finite
 897 almost surely, and Algorithm 4 terminates with probability 1.

898 *Expected query count and tail bound.* Let R_k be the number of uncollected secret
 899 indices after k effective queries, and let $r = R_{k-1} > 0$. Each of the w hidden
 900 leaves (recall $v^* = \text{root}$, so all w hidden leaves are exposed) independently draws
 901 a uniform index from $\{1, \dots, s-1\}$, so the probability that *none* of the w draws

falls in the remaining set of size r is $\left((s-1-r)/(s-1)\right)^w$. Hence the probability of revealing at least one new index is

$$p_r = 1 - \left(\frac{s-1-r}{s-1}\right)^w = 1 - \left(1 - \frac{r}{s-1}\right)^w. \quad (5)$$

Since $1 - r/(s-1)$ is decreasing in r , we have $p_r \geq p_{\min} = 1 - (1 - 1/(s-1))^w$ for all $r \geq 1$.

The expected total number of effective queries is

$$\mathbb{E}[Q] = \sum_{r=1}^m \frac{1}{p_r} \leq \frac{m}{p_{\min}} = \frac{s-1}{p_{\min}}, \quad (6)$$

where the inequality uses $p_r \geq p_{\min}$ for all $r \geq 1$.

Remark 9. The bound in (6) should be seen as an upper-bound, as it treats the recovery process as just learning one secret per time. In practice, a query may reveal multiple unseen indices simultaneously, especially during the early stages of the algorithm when r is large. As a consequence the actual expected number of queries can be smaller than $(s-1)/p_{\min}$.

For the tail bound, each $T_j \leq k_0$ with probability at least $1 - (1 - p_{\min})^{k_0}$ (geometric tail), so by a union bound over $m = s-1$ stages,

$$\Pr[Q > k] \leq (s-1) \cdot (1 - p_{\min})^{\lfloor k/(s-1) \rfloor}.$$

Setting $(s-1)(1 - p_{\min})^{\lfloor k/(s-1) \rfloor} \leq \delta$ and solving gives $k \leq \left\lceil \frac{s-1}{p_{\min}} \cdot \ln \frac{s-1}{\delta} \right\rceil$, establishing the bound in Proposition 4.

Output correctness. When the loop terminates, $|\text{Collected}| = m$, and for each $j \in \{1, \dots, s-1\}$ there is exactly one entry $(j, \text{sk}'_j) \in \text{Collected}$ with $\text{sk}'_j = A_j^{-1}$ (or the corresponding secret for each scheme). The Collect step assembles these into $\widehat{\text{sk}}$, which equals sk by correctness of Extract.

A.5 Derivation of Proposition 5 (Expected query count)

Setup. We work in the root-fault setting of Proposition 5: $v^* = \text{root}$, so $\ell = t$ and $L = w$ exactly (deterministic). By the Fiat-Shamir uniformity assumption, the challenge index c_i is uniform in $\{1, \dots, s-1\}$ independently for each $i \in \mathcal{H}$.

Distinct secrets per query. Conditioning on $L = w$, the number of *distinct* secret indices revealed in one effective query has expectation

$$\mathbb{E}[\#\text{distinct} \mid L = w] = (s-1) \left(1 - \left(1 - \frac{1}{s-1} \right)^w \right) = E_{\text{dist}}, \quad (7)$$

by linearity of expectation and symmetry of the uniform distribution.

928 *Remark 10 (General fault location).* When $v^* \neq \text{root}$ and L is a hypergeometric
 929 random variable with mean $\bar{w} = w\ell/t$, taking expectation over L gives

$$\mathbb{E}[\#\text{distinct}] = (s-1) \left(1 - \mathbb{E} \left[\left(1 - \frac{1}{s-1} \right)^L \right] \right).$$

930 Since $f(x) = \alpha^x$ with $\alpha = 1 - 1/(s-1) \in (0, 1)$ is convex, Jensen's inequality
 931 yields $\mathbb{E}[f(L)] \geq f(\bar{w}) = \alpha^{\bar{w}}$; hence

$$\mathbb{E}[\#\text{distinct}] \leq (s-1) \left(1 - \left(1 - \frac{1}{s-1} \right)^{\bar{w}} \right) =: E_{\text{dist}}^{\bar{w}}. \quad (8)$$

932 $E_{\text{dist}}^{\bar{w}}$ is an *upper* bound on the expected per-query gain, which implies a *lower*
 933 bound on $\mathbb{E}[Q]$. It cannot be used to derive an upper bound on $\mathbb{E}[Q]$; the upper
 934 bound in Proposition 5 follows solely from the direct inequality $p_r \geq p_{\min}$
 935 established in the root-fault setting.

936 *Total queries (root fault).* From equation (5) (see page 31), $p_r = 1 - (1 - r/(s -$
 937 $1))^w$ (with $L = w$ deterministic), and since $p_r \geq p_{\min}$ for all $r \geq 1$,

$$\mathbb{E}[Q] = \sum_{r=1}^m \frac{1}{p_r} \leq \frac{m}{p_{\min}} = \frac{s-1}{p_{\min}},$$

938 which for $p_{\min} \approx 1$ (all MEDS parameter sets, cf. Table 2) gives $\mathbb{E}[Q] \lesssim s-1$.